Organizing Your Robot Project in a Single File

Organizing Your Robot Project in a Single File

In this course, your robot programs will grow from simple scripts into projects with multiple classes working together. This guide shows you how to keep everything organized in **one file** so it is easy to work on, easy to test, and easy to turn in.

Why one file?

- **Your IDE works better.** Some editors have trouble when classes are split across separate files. Keeping everything together avoids those headaches.
- **Easier to turn in.** You submit one file per lesson or module instead of a folder of files. Your teacher can open it and immediately see all of your work.
- **Easier to run on the robot.** You copy one file to the XRP and run it. No worrying about which files need to be uploaded together.

The pattern

Your file will have three sections, always in this order:

```
1. Classes          (at the top)
2. Task functions   (in the middle)
3. Run section      (at the bottom)
```

Here is what each section does.

Section 1 – Classes

Put all of your classes at the top of the file. Earlier classes go first, because later classes may use them.

```python
from XRPLib.reflectance import Reflectance
from XRPLib.differential_drive import DifferentialDrive
from XRPLib.board import Board
import time



# ===== LineSensor Class =====


class LineSensor:
    def __init__(self):
        self.reflectance = Reflectance.get_default_reflectance()
        self.threshold = 0.5

    def get_left(self):
        return self.reflectance.get_left()

    def get_right(self):
        return self.reflectance.get_right()

    def get_error(self):
        return self.get_left() - self.get_right()

    def is_at_cross(self):
        return self.get_left() > self.threshold and self.get_right() >
        self.threshold

    def is_off_line(self):
        return self.get_left() < self.threshold and self.get_right() <
        self.threshold



# ===== LineTrack Class =====
```

```python
class LineTrack:
    def __init__(self):
        self.sensor = LineSensor()
        self.drivetrain = DifferentialDrive.get_default_differential_drive()
        self.base_effort = 0.4
        self.Kp = 0.5

    def track_until_cross(self):
        while not self.sensor.is_at_cross():
            error = self.sensor.get_error()
            correction = error * self.Kp
            self.drivetrain.arcade(self.base_effort, -correction)
        self.drivetrain.stop()

    # ... turn_right, turn_left, etc.
```

Tip: Use the `# ===== Class Name =====` comment lines to make it easy to scroll through the file and find each class.

Section 2 – Task functions

Below your classes, write a **function for each task** you are asked to complete. Name each function after the task so you (and your teacher) can find it quickly.

```python
# ==============================================================
#                        TASK FUNCTIONS
# ==============================================================

def task_1_test_line_sensor():
    """Task 1: Read sensor values and print them."""
    sensor = LineSensor()
    board = Board.get_default_board()
    board.wait_for_button()
    for i in range(10):
        print("Left:", sensor.get_left(), "Right:", sensor.get_right())
        time.sleep(0.5)


def task_2_follow_line():
    """Task 2: Follow a line until a cross is detected."""
    tracker = LineTrack()
    board = Board.get_default_board()
    board.wait_for_button()
    tracker.track_until_cross()
    print("Cross found!")


def task_3_follow_and_turn():
    """Task 3: Follow a line, turn right at the cross, follow again."""
    tracker = LineTrack()
    board = Board.get_default_board()
    board.wait_for_button()
    tracker.track_until_cross()
    tracker.turn_right()
```

```
    tracker.track_until_cross()
    print("Done!")
```

Each function is **self-contained** – it creates the objects it needs, does the work, and prints results. This means you can run any task by itself without affecting the others.

Section 3 – Run section

At the very bottom of the file, call the function for the task you are currently working on. When you finish a task and move to the next one, comment out the old call and add the new one.

```
# ============================================================
#                          RUN SECTION
# ============================================================
# Uncomment the task you want to run. Only run one at a time.

# task_1_test_line_sensor()
# task_2_follow_line()
task_3_follow_and_turn()
```

Only one task should be uncommented at a time. This way, when you press Run, Python executes just the task you are working on.

A complete example

Here is what a finished file looks like with everything put together:

```python
from XRPLib.reflectance import Reflectance
from XRPLib.differential_drive import DifferentialDrive
from XRPLib.board import Board
import time



# ===== LineSensor Class =====


class LineSensor:
    def __init__(self):
        self.reflectance = Reflectance.get_default_reflectance()
        self.threshold = 0.5

    def get_left(self):
        return self.reflectance.get_left()

    def get_right(self):
        return self.reflectance.get_right()

    def get_error(self):
        return self.get_left() - self.get_right()

    def is_at_cross(self):
        return self.get_left() > self.threshold and self.get_right() >
        self.threshold

    def is_off_line(self):
        return self.get_left() < self.threshold and self.get_right() <
        self.threshold



# ===== LineTrack Class =====
```

```python
class LineTrack:
    def __init__(self):
        self.sensor = LineSensor()
        self.drivetrain = DifferentialDrive.get_default_differential_drive()
        self.base_effort = 0.4
        self.Kp = 0.5

    def track_until_cross(self):
        while not self.sensor.is_at_cross():
            error = self.sensor.get_error()
            correction = error * self.Kp
            self.drivetrain.arcade(self.base_effort, -correction)
        self.drivetrain.stop()

    def turn_right(self):
        self.drivetrain.straight(8)
        self.drivetrain.turn(90)
        while self.sensor.is_off_line():
            self.drivetrain.arcade(0, 0.3)
        self.drivetrain.stop()

    def turn_left(self):
        self.drivetrain.straight(8)
        self.drivetrain.turn(-90)
        while self.sensor.is_off_line():
            self.drivetrain.arcade(0, -0.3)
        self.drivetrain.stop()



# ==============================================================
#                        TASK FUNCTIONS
# ==============================================================
```

```python
def task_1_test_line_sensor():
    """Task 1: Read sensor values and print them."""
    sensor = LineSensor()
    board = Board.get_default_board()
    board.wait_for_button()
    for i in range(10):
        print("Left:", sensor.get_left(), "Right:", sensor.get_right())
        time.sleep(0.5)


def task_2_follow_line():
    """Task 2: Follow a line until a cross is detected."""
    tracker = LineTrack()
    board = Board.get_default_board()
    board.wait_for_button()
    tracker.track_until_cross()
    print("Cross found!")


def task_3_follow_and_turn():
    """Task 3: Follow a line, turn right at the cross, follow again."""
    tracker = LineTrack()
    board = Board.get_default_board()
    board.wait_for_button()
    tracker.track_until_cross()
    tracker.turn_right()
    tracker.track_until_cross()
    print("Done!")


# ============================================================
```

```
#                          RUN SECTION
# ============================================================
# Uncomment the task you want to run. Only run one at a time.

# task_1_test_line_sensor()
# task_2_follow_line()
task_3_follow_and_turn()
```

Adding new classes later

As the course progresses, you will add new classes (like `Manhattan` and `Navigator`). When you do:

1. Add the new class **below** the existing classes in Section 1.
2. Write a new task function in Section 2 that uses the new class.
3. Update the Run section to call your new task function.

Your earlier classes and task functions stay in the file. You can always go back and run an earlier task to re-test something.

Quick-reference checklist

- All `import` statements are at the very top
- All classes are in Section 1, in the order they were created
- Each class has a `# ===== Class Name =====` header comment
- Each task is a separate function in Section 2
- Task functions are named `task_N_short_description()`
- Only one task is uncommented in the Run section

The file runs without errors when you press Run